

预备知识 P03: 概率与信息论够用版——softmax / 似然 / 熵 / 交叉熵 / KL

LLM 训练里反复出现的几个数学符号——softmax、 $\log p(x)$ 、熵 H 、交叉熵 $H(p, q)$ 、KL 散度 $D_{\text{KL}}(p \parallel q)$ ——背后是同一套概率与信息论语言。把它讲清楚之后，再回头看：

- 「下一个 token 预测的训练目标」 = 极大似然估计 (MLE) = 最小化交叉熵
- 「PPO / DPO 里的 KL 约束」 = 让新策略不要偏离参考分布太远
- 「temperature / top-p 采样」 = 在 softmax 出来的分布上做不同方式的采样

这些看似不同的招式，其实用同一套概率工具就能说穿。这一章把这套工具按"看公式 → 最小数值例子 → 用 PyTorch 验证"三步走一遍，不堆推导，只讲够用。

想直接跑示例？点这里  [Open in Colab](#)。

硬件门槛：概念章，CPU 即可✓。本章只算几个 3 ~ 5 维向量上的小例子，CPU 一秒跑完。

一、概率与信息论和模型训练的关系

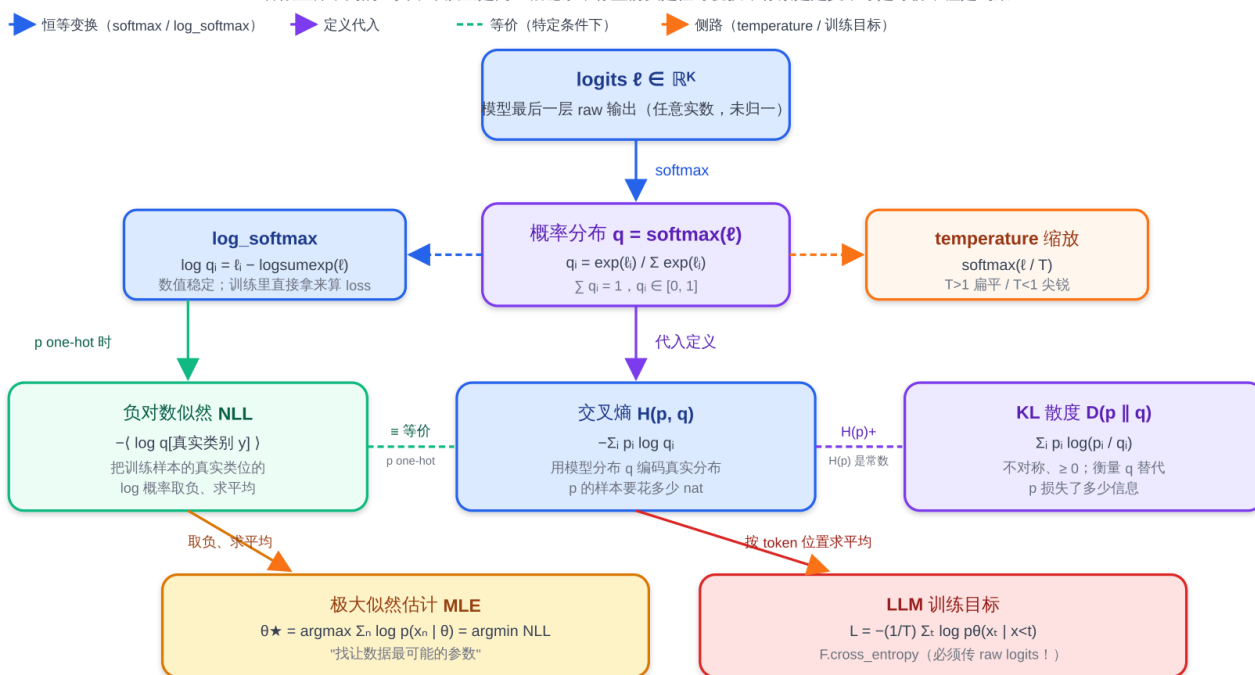
如果跳过这一章直接读 attention 实现 / loss 写法 / RLHF 论文，几乎一定会卡在下面这几个地方：

- 看到 `F.cross_entropy(logits, target)` (计算交叉熵损失) 不知道为什么要传 raw logits 而不是 softmax 出来的概率
- 看到论文里 $\mathcal{L} = -\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)]$ (数据分布下的负对数似然，即训练损失)，说不清这跟训练循环里调用的 `loss = ...` 是同一件事
- 看到 PPO / DPO 里 $\beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ (KL 惩罚项，约束新策略不偏离参考分布)，不知道这一项为什么能"防止跑偏"
- 看到 "temperature 调高让分布更扁平"，知道结论但说不清数学上发生了什么

把概率分布、似然、熵、交叉熵、KL 五个概念串起来，上面每个问题都能用一两句话说清楚。下面这张「概念关系图」把这一章要讲的所有工具按"链条"摆一遍——读完后每一节再回头看这张图，所有箭头都会变得显然：

概率与信息论概念关系图 —— logits / softmax / NLL / 交叉熵 / KL / MLE

看似五样不同的工具，本质上是同一根链条；标蓝箭头是恒等变换，标紫是定义，绿是等价，橙是约束



RLHF / PPO / DPO 里见到的 $\beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$:
让对齐的策略 π_{θ} 不要离参考 π_{ref} 太远 —— 这是上面 KL 框的“约束”用法 (reverse KL, mode-seeking)

最值得记的几条: logits $\rightarrow$ softmax $\rightarrow$ 概率 q 是恒等链; MLE = min NLL = min 交叉熵 (在 one-hot 标签下); 最小化交叉熵 $\equiv$ 最小化 KL (当真实分布固定)。RLHF 里的 $\beta \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ 是同一根 KL 在“约束”位置的另一种用法。

二、概率分布：从 logits 到 softmax

2.1 离散分布与 Categorical

最常见的离散概率分布——Categorical 分布 (中文「类别分布」, 又叫分类分布) ——长这样:

$$p = (p_1, p_2, \dots, p_K), \quad p_i \geq 0, \quad \sum_{i=1}^K p_i = 1$$

K 是类别数 (对 LLM 来说就是词表大小, 常见在 5 万 ~ 15 万)。每个 p_i 是“取到第 i 类”的概率。从这个分布里采样就是按权重 p_i 抽一个类别 id。

最小例子 ($K = 3$) :

$p = (0.7, 0.2, 0.1)$
↑ 七成概率取第 0 类、两成第 1 类、一成第 2 类

LLM 推理时每一步 forward 输出的就是这种长度为 K 的概率向量, `top-k` / `top-p` / `temperature` 都是在这个向量上做手脚。

2.2 logits 是什么

模型最后一层（线性投影）的输出是一条任意实数向量：

$$\ell = (\ell_1, \ell_2, \dots, \ell_K) \in \mathbb{R}^K$$

这就是 logits——「未归一化的分数」。它的特征是：

- 可以是任意实数（正、负、几十、几百都行）
- 大小关系已经决定了"模型最看好哪一类"——`argmax` 就是贪心解码的结果
- 但它还不是概率——既不在 $[0, 1]$ 内，也不归一

要把 logits 变成概率，得过一道 softmax。

2.3 softmax 的定义与性质

$$\text{softmax}(\ell)_i = \frac{\exp(\ell_i)}{\sum_{j=1}^K \exp(\ell_j)}$$

它有四条值得记的性质：

1. 输出非负且和为 1——是合法的概率分布
2. 保单调性： $\ell_i > \ell_j \Rightarrow p_i > p_j$ ，所以"最大 logit"就是"最大概率"
3. 平移不变性： $\text{softmax}(\ell + c) = \text{softmax}(\ell)$ ，给所有 logit 加同一个常数不变；这是数值稳定 softmax 的依据（见第 2.4 节）
4. 温度缩放： $\text{softmax}(\ell/T)$ ， $T > 1$ 让分布更扁平、 $T < 1$ 让分布更尖锐——这就是 LLM 推理里 `temperature` 参数背后的数学

最小数值例子（ $K = 3$ ）：

```
ℓ = (2.0, 1.0, 0.1)
exp(ℓ) = (7.389, 2.718, 1.105)
sum    = 11.213
softmax(ℓ) = (0.659, 0.242, 0.099)
```

注意 logit 差 1，`exp` 之后比例就拉到 $e \approx 2.72$ 倍——这就是为什么 logit 差几个点就能让某类概率压倒性地高。

2.4 数值稳定的 softmax 与 log-softmax

直接按公式算 $\exp(\ell_i)$ 时，如果 logit 很大（比如 100）， $\exp(100) \approx 2.7 \times 10^{43}$ 会浮点上溢；很负就下溢成 0。工程实现都用减最大值的等价版本（利用平移不变性）：

$$m = \max_j \ell_j, \quad \text{softmax}(\ell)_i = \frac{\exp(\ell_i - m)}{\sum_j \exp(\ell_j - m)}$$

减完 m 后所有指数项都 ≤ 1 ，不会溢出。 `torch.softmax` 内部就是这么实现的。

进一步，训练里几乎只用 `log-softmax`——直接得到 $\log p_i$ ，配合后面要讲的“取对数 + 取负”的损失。从 `softmax` 定义对两边取 $\log$ ，用 $\log \frac{a}{b} = \log a - \log b$ 一步化简：

$$\log \text{softmax}(\ell)_i = \log \frac{\exp(\ell_i)}{\sum_j \exp(\ell_j)} = \ell_i - \log \sum_j \exp(\ell_j)$$

把后面那一项单独命名（PyTorch 里就是 `torch.logsumexp`）：

$$\text{LSE}(\ell) \triangleq \log \sum_j \exp(\ell_j)$$

于是 `log-softmax` 就写成简洁形式

$$\log \text{softmax}(\ell)_i = \ell_i - \text{LSE}(\ell)$$

`LSE` 本身按定义算 $\exp(\ell_j)$ 也会上溢，所以工程实现走和 `softmax` 同款的“减最大值”等价式：

$$\text{LSE}(\ell) = m + \log \sum_j \exp(\ell_j - m), \quad m = \max_j \ell_j$$

直接在 logits 上用 `F.log_softmax` 比“先 `softmax` 再 `log`”数值稳定得多——后者两头都会坏：logits 偏大时 `exp` 直接上溢成 `inf`，`inf/inf` 让概率全变 `nan`；logits 相差悬殊时小概率被压成 0，再取 `log` 拿到 `-inf`。

延续第 2.3 节那个数值例子算一下 `log-softmax`：

```
ℓ = (2.0, 1.0, 0.1)
m = max(ℓ) = 2.0
ℓ - m          = (0.0, -1.0, -1.9)
exp(ℓ - m)      = (1.000, 0.368, 0.150)
sum exp(ℓ - m)  = 1.517
LSE(ℓ)          = m + log(sum exp(ℓ - m)) = 2.0 + 0.417 = 2.417
log-softmax(ℓ)  = ℓ - LSE(ℓ) = (-0.417, -1.417, -2.317)
```

可以验证一下 $\exp(-0.417) \approx 0.659$ 、 $\exp(-1.417) \approx 0.242$ 、 $\exp(-2.317) \approx 0.099$ ——和第 2.3 节里的 `softmax` 输出对得上。还可观察到：输入 ℓ 之间的差异（1.0、0.9）在 `log-softmax` 里原样保留（差仍是 1.0、0.9）——`LSE` 对每一位都减同一个常数，差值不变。

实战要点：训练时 forward 出 logits，配合 `F.cross_entropy` 或 `F.nll_loss(F.log_softmax(...))`，不要自己 `softmax` 再 `log`。

三、似然与极大似然估计 (MLE)

3.1 似然 vs 概率

同一个表达式 $p(x | \theta)$:

- 把 θ 看死、 x 当变量——叫概率 (probability) : 在给定参数下, 数据 x 出现的可能性
- 把 x 看死 (已观测)、 θ 当变量——叫似然 (likelihood) : 在观测到这份数据后, 参数 θ 有多"可信"

写法上经常用 $\mathcal{L}(\theta; x) = p(x | \theta)$ 强调"现在把它当 θ 的函数看"。

举个具体例子: 抛一枚硬币 10 次, 观察到 7 次正面、3 次反面。设正面概率为 θ , 每次抛掷独立同分布 (伯努利), 则"10 次里 7 正 3 反"这一观测的概率为:

$$p(x | \theta) = \binom{10}{7} \theta^7 (1 - \theta)^3$$

同一个表达式可以从两个角度看:

- 概率视角: 把 θ 钉死 (比如假设硬币公平、 $\theta = 0.5$) , 问"10 次里 7 正"这种结果出现的可能性——
 $p(x | \theta = 0.5) = \binom{10}{7} \cdot 0.5^{10} \approx 0.117$ 。变量是 x , θ 是已知参数。
- 似然视角: 硬币已经抛完, 观测 x 不再是变量; 把同一表达式当作 θ 的函数 $\mathcal{L}(\theta) = \binom{10}{7} \theta^7 (1 - \theta)^3$, 问"哪个 θ 最能解释这份观测"。对 $\log \mathcal{L}(\theta)$ 求导令为 0, 解出 $\hat{\theta} = 7/10 = 0.7$ ——这就是 MLE 给出的"最可信"参数估计 (详见第 3.3 节)。

一句话总结: 同一个 $p(x | \theta)$, x 是变量就叫概率, θ 是变量就叫似然。

3.2 为什么取对数: log-likelihood

假设观测到 N 条独立同分布 (i.i.d.) 的样本 $x_1, \dots, x_N$, 整体似然是连乘:

$$\mathcal{L}(\theta) = \prod_{n=1}^N p(x_n | \theta)$$

为什么是连乘? 靠 i.i.d. 这两个定语:

- 独立——根据独立性的定义, 多个独立随机变量的联合概率等于各自概率的乘积:
 $p(x_1, x_2, \dots, x_N | \theta) = p(x_1 | \theta) \cdot p(x_2 | \theta) \cdots p(x_N | \theta)$
- 同分布——每个 $p(x_n | \theta)$ 是同一个表达式、共用同一个 θ ; 不会出现"第 1 条用 θ 、第 2 条用 θ' "的情况

回到第 3.1 节抛硬币的例子: 把"抛 10 次"看成 10 个独立的伯努利样本, 每次出正面的概率都是同一个 θ ; 某条具体序列 (如"正反正正反正正正") 的概率就是 $\theta^7 (1 - \theta)^3$ ——同一个 θ 在 10 个因子里相乘。组合数 $\binom{10}{7}$ 只是把"7 正 3 反"所有可能顺序合并起来, 连乘的本质没有消失。

直接算这个连乘在工程上有两条致命问题：

1. 数值下溢：每个 $p(x_n)$ 通常很小（ 10^{-3} 量级），连乘 1000 项就成了 10^{-3000} ，浮点直接归零
2. 不便求导：连乘求导得用积法则，链上展开非常困难

取一次对数就两个问题一起解决：

$$\log \mathcal{L}(\theta) = \sum_{n=1}^N \log p(x_n | \theta)$$

连乘变连加，下溢风险消失（ $\log 10^{-3000} = -3000 \log 10 \approx -6907$ ，普通浮点装得下），求导变成对每一项分别求导。绝大多数训练代码里 loss 都是 log-likelihood 的形式。

3.3 MLE：训练即"找让数据最可能的参数"

极大似然估计（Maximum Likelihood Estimation, MLE）的核心：

$$\theta^* = \arg \max_{\theta} \log \mathcal{L}(\theta) = \arg \max_{\theta} \sum_{n=1}^N \log p(x_n | \theta)$$

习惯上把"max log-likelihood"翻成等价的"min 负对数似然（NLL）"，以便走优化器的"min loss"惯例：

$$\mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{N} \sum_{n=1}^N \log p(x_n | \theta)$$

LLM 的"下一个 token 预测"训练目标几乎一字不差：

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$$

—— x_t 是序列里第 t 个 token， $p_{\theta}(\cdot | x_{<t})$ 是模型在已生成前 $t-1$ 个 token 的条件下输出的下一个 token 分布（softmax 出来的概率向量）， T 是序列长度。模型每一步把"真实下一个 token 的概率"提到尽量高。

数值例子（迷你 token 预测，数据是模拟的）：把上面那个"每一步"单独抠出来算一遍。设词表只有 3 个 token $\{A, B, C\}$ （想成 {好, 差, 热} 这类即可）。训练语料里"今天天气真 ____"这个 context 共出现 10 次，标注的真实下一个 token 是：

- A 出现 7 次
- B 出现 2 次
- C 出现 1 次

模型在这个 context 下吐出的 softmax 概率向量记作 $\theta = (\theta_A, \theta_B, \theta_C)$ ，归一化约束 $\theta_A + \theta_B + \theta_C = 1$ 。把这 10 个位置当独立观测，对数似然就是：

$$\log \mathcal{L}(\theta) = 7 \log \theta_A + 2 \log \theta_B + 1 \log \theta_C$$

——按出现次数加权求和"真实下一个 token 的对数概率"——这正是 LLM 训练 loss 在一个 context 下展开的样子。在归一化约束下用 Lagrange 乘子最大化它，可解出：

$$\hat{\theta} = (0.7, 0.2, 0.1)$$

——正好等于经验频率（每个 token 的计数除以总数），是第 3.1 节伯努利 7 正 3 反的多类推广。

代几个候选分布对照看一眼：

θ	含义	$\log \mathcal{L}(\theta)$
$(1/3, 1/3, 1/3)$	均匀分布（模型什么都没学到）	$10 \log(1/3) \approx -10.99$
$(0.7, 0.2, 0.1)$	MLE 解	$7 \log 0.7 + 2 \log 0.2 + 1 \log 0.1 \approx -8.02$
$(0.1, 0.2, 0.7)$	把高频 token 的概率压低（错方向）	$7 \log 0.1 + 2 \log 0.2 + 1 \log 0.7 \approx -19.69$

把对数似然取负、再除样本数 $N = 10$ ，就是 P02 训练循环里那个"平均 NLL"（单位 nat）：MLE 解最低 ≈ 0.802 ，均匀分布次之 ≈ 1.099 ，把高频 token 概率压低的最差 ≈ 1.969 。"让每个真实下一个 token 的概率尽量高"和"让平均 NLL 尽量低"是同一件事——这就是 LLM 训练 loss 每一步都在干的活，只是真实词表是几万维、context 也不再固定为一条，而是序列里每个位置随 $x_{<t}$ 变化。

一句话总结训练目标：通过反向传播 + 梯度下降不断更新参数 θ 的方式，让对数似然 $\log \mathcal{L}(\theta)$ 最大，也就是让平均 NLL（在 one-hot 标签下等于 cross-entropy loss）最小。

下一节会看到，这个目标函数还能从"信息论的交叉熵"角度等价推出。

四、熵：不确定性的度量

一个分布 $p = (p_1, \dots, p_K)$ 的熵（entropy）：

$$H(p) = - \sum_{i=1}^K p_i \log p_i$$

约定 $0 \cdot \log 0 = 0$ 。理由是取极限 $\lim_{x \rightarrow 0^+} x \log x = 0$ —— x 趋于 0 比 $\log x$ 趋于 $-\infty$ 快，乘积收敛到 0。直觉上也合理：概率为 0 的事件根本不会发生，对"平均不确定性"的贡献理应是 0。所以遇到 $p_i = 0$ 的项直接当 0 跳过，整个熵仍是良定义的有限值。

直觉是「编码这个分布的样本平均要花多少比特」——也是「这个分布有多不确定」：

分布	熵	直觉
一点全押（如 $(1, 0, 0, 0)$ ）	$H = 0$	完全确定，没有信息

分布	熵	直觉
均匀分布（ K 类各 $1/K$ ）	$H = \log K$	最不确定，熵达到上界
介于两者之间	$0 < H < \log K$	越尖越低、越平越高

`log` 用 2 为底单位是 bit，自然对数是 nat。深度学习里默认 nat（PyTorch 全用自然对数）。

最小数值例子：

p	$H(p)$ (nat)	$H(p)$ (bit)
(1, 0, 0)	0	0
(0.5, 0.5, 0)	$\log 2 \approx 0.693$	1
(1/3, 1/3, 1/3)	$\log 3 \approx 1.099$	$\log_2 3 \approx 1.585$

熵在 LLM 里直接出现的场景：

- 采样温度的效果可以用熵衡量——温度高、分布扁、熵高；温度低、分布尖、熵低
- 训练后期模型在某个 token 上的熵很低——说它"很笃定"
- PPO / RLHF 经常加一项熵奖励鼓励策略多样性，防止过早收敛到单一动作

五、交叉熵：模型分布离真实分布有多远

5.1 定义

设真实分布是 p 、模型预测的分布是 q ，交叉熵（cross-entropy）：

$$H(p, q) = - \sum_{i=1}^K p_i \log q_i$$

注意里面 $\log$ 是套在 q 上、外面权重用 p ——「用模型分布 q 对真实分布 p 的样本编码要花多少比特」。可以证明 $H(p, q) \geq H(p)$ ，等号当且仅当 $p = q$ ——所以最小化交叉熵等价于让 q 逼近 p 。

为什么这么定义？拆开看每一项：

- $-\log q_i$ ：看到类别 i 时，按模型 q 估计这件事的"惊讶程度"—— q_i 越小越意外，取负对数就越大
- p_i ：类别 i 在真实世界里有多常出现
- $\sum_i p_i \cdot (-\log q_i)$ ：把每一类的"惊讶值"按真实频率加权平均—— "在真实分布下，用模型 q 看世界的平均惊讶程度"

直觉上：模型 q 哪里和真相 p 偏得越多，那里就越意外；而越意外的地方如果偏偏是真实里常发生的（ p_i 大），惩罚就越重。

最小数值例子：取真实分布 $p = (0.7, 0.2, 0.1)$ ，比较三种候选 q （单位 nat）：

候选 q	$-\log q_1$	$-\log q_2$	$-\log q_3$	$H(p, q) = \sum_i p_i \cdot (-\log q_i)$
$q_a = (0.7, 0.2, 0.1)$ ，完美匹配	0.357	1.609	2.303	$0.7 \cdot 0.357 + 0.2 \cdot 1.609 + 0.1 \cdot 2.303 \approx 0.802$
$q_b = (0.5, 0.3, 0.2)$ ，差不多	0.693	1.204	1.609	$0.7 \cdot 0.693 + 0.2 \cdot 1.204 + 0.1 \cdot 1.609 \approx 0.887$
$q_c = (0.1, 0.2, 0.7)$ ，严重错位	2.303	1.609	0.357	$0.7 \cdot 2.303 + 0.2 \cdot 1.609 + 0.1 \cdot 0.357 \approx 1.969$

三件事一眼看到：

- 1. $q_a = p$ 时取到下界 $H(p) \approx 0.802$ 纳特——这正是"真实分布自身的熵"，再小不下去了
- 2. q_b 偏一点，交叉熵就抬高一点（ $0.887 > 0.802$ ）——多出来的 0.085 纳特就是"用 q_b 替代 p 多花的代价"，也就是后面第 6.2 节要讲的 KL 散度 $D_{KL}(p \parallel q_b)$
- 3. q_c 把概率压在了真实里很少出现的类别上，于是 $-\log q_1 = 2.303$ 这一大项被 $p_1 = 0.7$ 这个大权重狠狠放大，交叉熵直接飙到 1.969 ——这就是「模型在高频真实事件上越不自信，惩罚越重」

如果再极端一点令 $q_1 \rightarrow 0$ ，那 $-\log q_1 \rightarrow +\infty$ ，交叉熵也会爆炸——直觉是「模型咬定真实里 70% 都出现的事件根本不可能发生，这种错误是无限糟糕的」。这条性质在训练中体现为"模型不能给真实 token 分配 0 概率"，也是 label smoothing 等技巧背后的动机之一。

5.2 与负对数似然（NLL）的等价性

如果训练数据每条样本都是一个确定的类别（one-hot 标签），那真实分布就是 $p = (\dots, 0, 1, 0, \dots)$ ——只有真实类别那一位是 1。代入交叉熵：

$$H(p, q) = - \sum_i p_i \log q_i = - \log q_{\text{真实类别}}$$

——只剩下"对真实类别那位的 $-\log q$ "。再对所有样本求平均（把第 n 条样本的真实类别 id 记作 $y_n \in \{1, \dots, K\}$ ， q_{n, y_n} 就是模型 q_n 在该样本真实类别那一位上的概率）：

$$\frac{1}{N} \sum_n H(p_n, q_n) = - \frac{1}{N} \sum_n \log q_{n, y_n} = \mathcal{L}_{\text{NLL}}$$

所以「最小化交叉熵 loss」和「最大化 log-likelihood / 最小化 NLL」是同一件事。LLM 训练里的 cross-entropy 就是把这条公式按 token 位置一份一份算出来的（此时 y_n 换成"第 n 个位置上真实出现的下一个 token id"）。

关于 one-hot 的一个常见误解：one-hot 描述的是单条样本的标签结构（某位为 1、其余为 0），并不要求“同一前缀下的下一个 token 唯一”。回到第 3.3 节的例子——「今天天气真 ____」出现 10 次，标注分别是 7 个 A、2 个 B、1 个 C——这 10 条样本各自的标签仍然是 one-hot（7 条 (1,0,0)、2 条 (0,1,0)、1 条 (0,0,1)），自然语言里“同前缀对应多种延续”这件事并不破坏单样本的 one-hot 性质。把 10 条样本的 NLL 求和 $-(7\log q_A + 2\log q_B + \log q_C)$ ，数值上等于“与经验分布 $\hat{p} = (0.7, 0.2, 0.1)$ 算的交叉熵 $H(\hat{p}, q)$ ”乘以 10——这是聚合视角，此时 $\hat{p}$ 自身已不是 one-hot。两种视角等价：模型在大量 one-hot 单样本上同时最小化 NLL，最优解会逼近经验频率分布——这正是 LLM 在 one-hot 标签下仍能学到软概率分布的原因。

5.3 F.cross_entropy 输入是 raw logits

把 5.2 的公式落到 PyTorch 上就是一行：

```
# input 是 raw logits (N, K); target 是真实类别 id (N,)
loss = F.cross_entropy(logits, target)
# 内部等价: F.nll_loss(F.log_softmax(logits, dim=-1), target)
```

注意 `F.cross_entropy` 期望传入 raw logits——它内部自己 `log_softmax`，手动 softmax 再传不会报错但梯度对应错误目标。这条工程坑及其数值稳定性原理（log-sum-exp）在 P02 也展开过，可对照阅读。

六、KL 散度：两个分布之间的差

6.1 定义

KL 散度（Kullback–Leibler divergence）：

$$D_{\text{KL}}(p \parallel q) = \sum_{i=1}^K p_i \log \frac{p_i}{q_i}$$

它衡量“用 q 替代 p 损失了多少信息”。性质：

1. $D_{\text{KL}}(p \parallel q) \geq 0$ ，等号当且仅当 $p = q$
2. 不对称： $D_{\text{KL}}(p \parallel q) \neq D_{\text{KL}}(q \parallel p)$ ——所以严格说不是“距离”
3. 不满足三角不等式

6.2 KL 与交叉熵的关系

把交叉熵展开：

$$H(p, q) = - \sum_i p_i \log q_i = - \sum_i p_i \log p_i + \sum_i p_i \log \frac{p_i}{q_i} = H(p) + D_{\text{KL}}(p \parallel q)$$

交叉熵 = 真实分布的熵 + KL 散度。当真实分布 p 固定时（例如训练数据已给定）， $H(p)$ 是常数，最小化交叉熵就是最小化 KL 散度。这把"用 cross-entropy 做训练目标"和"逼近数据分布"这两件事正式等同起来。

这条分解也顺便给出了 $D_{KL}(p \parallel q) \geq 0$ 的直觉：交叉熵 $H(p, q)$ 是「用模型 q 描述真实 p 的样本平均要花多少代价」，而 $H(p)$ 是「真实 p 自身不可压缩的最低代价」——用别的分布替代真实分布，代价只会更高、不会更低，所以多出来的差额 $D_{KL}(p \parallel q) = H(p, q) - H(p)$ 非负；只有当 q 完美匹配 p 时这笔差额才归零。回到第 5.1 节的数值例子也能直接验证：当 $q_a = p$ 时 $H(p, q_a) = H(p) \approx 0.802$ ， $KL = 0$ ；偏成 q_b 后 $H(p, q_b) \approx 0.887$ ，多出来的 0.085 纳特就是 $D_{KL}(p \parallel q_b)$ 。

6.3 不对称性

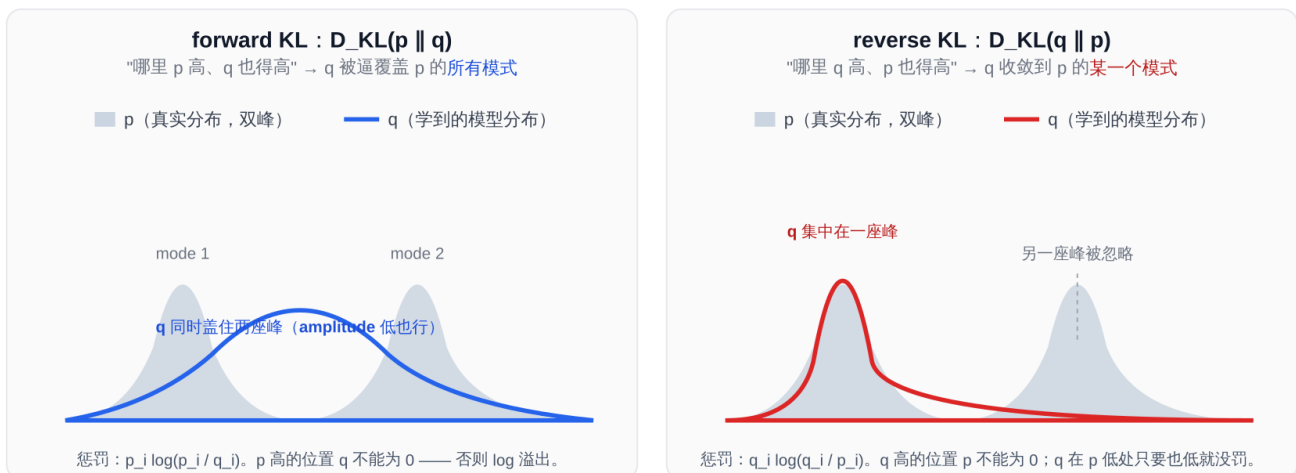
$D_{KL}(p \parallel q)$ 跟 $D_{KL}(q \parallel p)$ 在优化时倾向不同：

- forward KL $D_{KL}(p \parallel q)$: $p_i > 0$ 的地方 q_i 不能为 0——否则 $q_i \rightarrow 0$ 时 $p_i \log(p_i/q_i) \rightarrow +\infty$ ，惩罚极重——逼 q 覆盖 p 的全部支撑 (mode-covering)
- reverse KL $D_{KL}(q \parallel p)$: $p_i = 0$ 的地方 q_i 必须为 0 (zero-forcing) ——否则 $p_i \rightarrow 0$ 时 $q_i \log(q_i/p_i) \rightarrow +\infty$ ，惩罚极重——逼 q 缩进 p 的支撑里；当 q 容量不够（如单峰高斯拟合双峰 p ）时退化为只挑一个模式 (mode-seeking)

下图把双峰真实分布 p 与单峰模型分布 q 在两种 KL 下分别"长成什么样"画在一起：

forward KL vs reverse KL : mode-covering 还是 mode-seeking

真实分布 p (双峰，灰色) 固定；模型 q (蓝色) 选用单峰高斯，看它最终长成什么样



工程结果：SFT/预训练 = forward KL (覆盖训练数据多样性)；RLHF/DPO 的 $\beta \cdot D_{KL}(\pi_\theta \parallel \pi_{ref})$ 是 reverse KL (逼新策略不要离参考太远)

注意一个常见误解：mode-covering / mode-seeking 这组标签描述的是 q 容量不够（被参数化成单峰高斯，没法同时覆盖两个峰）时的退化行为，并不是 KL 本身的本质偏好。如果 q 有能力变双峰，两种 KL 的全局最优解都是 $q = p$ ——双峰都覆盖。reverse KL 真正本质的性质是 zero-forcing：哪里 $p_i = 0$ 就强迫 $q_i = 0$ （否则 $q_i \log(q_i/p_i) \rightarrow +\infty$ ）。容量受限时这条性质退化成"不愿在 p 低概率区误放概率，干脆躲进一个峰"；容量充足时它只是要求 $\text{supp}(q) \subseteq \text{supp}(p)$ ，对峰数没意见。RLHF 里 π_θ 是大型 Transformer 容量近乎无

限，所以 $\beta \cdot D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ 实际起的作用更像"正则化"——防止 π_θ 在 π_{ref} 从不输出的地方乱放概率（reward hacking），而不是真把策略压成单一动作。

LLM 里两种 KL 都见得到：

- 预训练 / SFT 用 cross-entropy $\Leftrightarrow$ forward KL（数据是"真实分布" p ）—— 让模型覆盖训练数据的所有可能延续
- PPO / DPO 等对齐阶段 加约束 $\beta \cdot D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ —— 这是 reverse KL（ π_θ 当 "q"、 π_{ref} 当 "p"），逼新策略不要离参考模型太远，防止 reward hacking
- 变分推断 / 强化学习里的 reverse KL 倾向于产生更尖锐、模式集中的分布

数值实例（手算）：

```
p = (0.5, 0.5)
q = (0.9, 0.1)

D_KL(p // q) = 0.5 log(0.5/0.9) + 0.5 log(0.5/0.1)
               = 0.5 * (-0.588) + 0.5 * (1.609)
               ≈ 0.510 nat

D_KL(q // p) = 0.9 log(0.9/0.5) + 0.1 log(0.1/0.5)
               = 0.9 * (0.588) + 0.1 * (-1.609)
               ≈ 0.368 nat
```

—— 同一对 (p, q) ， $D_{\text{KL}}(p \parallel q) \neq D_{\text{KL}}(q \parallel p)$ 。

七、把这套工具映射到 LLM 训练

把这章学到的概念按"在 LLM 里怎么用"摆一遍：

概念	在 LLM 训练 / 推理里的位置
logits	最后一层 <code>lm_head</code> 的输出，shape <code>(B, L, V)</code> ， <code>V</code> 是词表大小
softmax	把 logits 变成下一个 token 的概率分布；推理采样、计算 loss 内部都会用
temperature	<code>softmax(logits / T)</code> ： $T > 1$ 扁平、 $T < 1$ 尖锐
log-softmax	训练时直接拿来算 <code>cross_entropy</code> / <code>nll_loss</code> ，避免数值爆炸
log-likelihood / NLL	「下一个 token 预测」的训练目标 = 序列每个位置上的 NLL 之和
熵 $H(p)$	衡量分布尖锐还是扁平；RL 中常加熵奖励防止策略坍缩
交叉熵 $H(p, q)$	PyTorch <code>F.cross_entropy</code> 实现的就是它；与 NLL 在 one-hot 标签下完全等价

概念	在 LLM 训练 / 推理里的位置
KL 散度	RLHF / DPO 里 $\beta \cdot D_{KL}(\pi_{\theta} \parallel \pi_{ref})$ 的"防跑偏"约束；变分推断中常用 reverse KL

记住几条工程铁律就不容易踩坑：

- 1. forward 出 raw logits, loss 用 `F.cross_entropy` ——永远不要先 softmax 再喂给 loss
- 2. 训练时用 `log_softmax` / `logsumexp` , 不要"softmax 之后再取 log"
- 3. KL 散度是不对称的——读 RL / 对齐论文时永远先看清楚他写的是 $D_{KL}(\pi \parallel \pi_{ref})$ 还是反过来

八、关键概念回顾

概念	一句话定义	LLM 里的典型用法
logits	最后一层 raw 输出, 任意实数	模型 forward 的最终结果, shape <code>(B, L, V)</code>
softmax	把实数向量归一化成概率分布	logits $\rightarrow$ next-token 概率
temperature	softmax 前给 logits 整体除以 T	控制采样尖锐 / 扁平
log-softmax	softmax 取 log, 数值稳定形式	训练 loss 内部、采样时打分
likelihood	把 $p(x \mid \theta)$ 当 θ 的函数	"数据有多支持这套参数"
NLL	负对数似然 $-\log p(x \mid \theta)$ 的均值	训练 loss 的标准形式
熵 $H(p)$	$-\sum p_i \log p_i$, 分布的"不确定性"	RL 的熵奖励、衡量分布尖锐度
交叉熵 $H(p, q)$	$-\sum p_i \log q_i$, 与 NLL 等价	<code>F.cross_entropy</code> 的内部公式
KL 散度	$\sum p_i \log(p_i/q_i)$, 不对称、 ≥ 0	RLHF / DPO 的 KL 约束、变分推断

九、本章小结

这一章把 LLM 训练里反复碰到的概率与信息论工具串讲了一遍：

- logits $\rightarrow$ softmax $\rightarrow$ 概率分布——「未归一化分数」到「合法概率」之间只差一个 softmax；数值稳定靠减最大值与 log-softmax
- MLE = 最大化 log-likelihood = 最小化 NLL——"下一个 token 预测"训练目标几乎逐字就是这条公式
- 熵 $H(p)$ 度量分布的不确定性；交叉熵 $H(p, q) = H(p) + D_{KL}(p \parallel q)$

- `F.cross_entropy` 接收 raw logits——这是 PyTorch 训练里被踩得最多的坑
- KL 散度不对称——forward KL 倾向覆盖、reverse KL 倾向集中；读对齐 / RL 论文时永远先看清楚谁是 p 谁是 q

ipynb 里的实战部分会把上面每个公式都用 PyTorch 重算一遍：手写 softmax 与 `torch.softmax` 一致性、温度调节如何改变熵、`F.cross_entropy` 与"自己 log-softmax + nll"一致性、把 softmax 喂给 loss 会出什么错、KL 不对称的可视化。

预告 P04：下一章把"得到 loss → 怎么用 grad 更新参数"的优化器侧讲透——SGD / Momentum / Adam / AdamW 的差别、学习率为什么是最重要的超参、warmup + cosine 这套 LLM 训练的默认调度配方。